

SPIRIT KNIGHT

Technical Documentation

Nguyễn Mạnh Cường

Vũ Việt Dũng

Trần Minh Tuấn Kiệt

Ngày 11 tháng 8 năm 2026

Mục lục

1	Giới thiệu	2
1.1	Tổng quan	2
1.2	Mục tiêu dự án	2
1.3	Sơ lược về cốt truyện và gameplay	2
2	Kiến trúc hệ thống	3
2.1	Tổng quan	3
2.2	Hệ thống mã nguồn	3
2.3	Class Main.java	4
2.3.1	Khởi tạo giao diện JavaFX	4
2.3.2	Khởi tạo hệ thống Input	5
2.3.3	Khởi tạo GameWorld và UIManager	5
2.3.4	Khởi tạo Game Loop	5
2.3.5	Khởi tạo cơ sở dữ liệu bất đồng bộ	5
3	Hệ thống GameWorld	6
3.1	Tổng quan	6
3.2	Kiến trúc tổng thể	6
3.3	Các thành phần chính	7
3.3.1	Các Manager	7
3.4	Quản lý trạng thái trò chơi	7
3.4.1	Chuyển đổi trạng thái	8
3.5	Chu trình cập nhật GameWorld	8
3.6	Cập nhật gameplay	9
3.7	Quản lý Player và Pet	10
3.7.1	Pet System	10
3.8	Hệ thống chiến đấu	10
3.8.1	Quản lý Bullet	10
3.8.2	Damage	11
3.9	Hệ thống kỹ năng và hiệu ứng chiến đấu	11
3.10	Quản lý Item	12
3.11	Hệ thống Room và Reward	12
3.12	Hệ thống vũ khí trong một lượt chơi	12
3.13	Khởi tạo một lượt chơi	13
3.14	Render thế giới	13
3.14.1	Y-Sorting	13
3.14.2	Render Pipeline	13
3.15	Camera	14

3.16	Hệ thống Save và Load	14
3.16.1	Database Worker	15
3.16.2	Cloud Save	15
3.16.3	Cloud Load	15
3.17	Auto Save	15
3.18	Quản lý Gold và Gem	15
3.19	Quản lý đồng thời	16
3.20	Quá trình Shutdown	16
3.21	Luồng hoạt động tổng quát	16
3.22	Kết luận	17

Chương 1

Giới thiệu

1.1 Tổng quan

Spirit Knight là một game nhập vai phát triển bằng Java và JavaFX.

1.2 Mục tiêu dự án

Mục tiêu của dự án là củng cố các kiến thức lập trình nâng cao, quản lí logic , xây dựng tư duy hướng đối tượng , kĩ năng làm việc nhóm , sử dụng git

1.3 Sơ lược về cốt truyện và gameplay

Lấy cảm hứng từ tựa game Soul Knight - Chillyroom . Dự án được phát triển bởi nhóm sinh viên và học viên cao học ngành công nghệ thông tin .

Bối cảnh bắt đầu từ một không gian quen thuộc với các Dev đó là một buổi đêm muộn đêm bàn máy. Trong cái đêm định mệnh ấy bạn nhận được một thông điệp lạ và lời thỉnh cầu đi giải cứu trái đất đang bị các thế lực đen tối dòm ngó. Bạn là người được chọn và hành trình của bạn sẽ bắt đầu sau cánh cổng portal

Chương 2

Kiến trúc hệ thống

2.1 Tổng quan

Game được chia thành nhiều hệ thống như Player, Enemy, Weapon, Pet, Buff, Item, Database và UI

2.2 Hệ thống mã nguồn

Game được chia thành các package đảm bảo đưa các class liên quan về 1 package

- Package **animation**: gồm các class quản lí các hiệu ứng của game như: player/quái sinh/chết, render floating text, các hiệu ứng Particle pixel (các loại hạt), đổ bóng nhân vật
- Package **buff**: gồm danh sách các buff của game, tính năng của từng loại trong các class riêng, Package nhỏ **effect** phụ trách các hiệu ứng gây ra bởi các buff khi tấn công trực tiếp lên kẻ thù - các hiệu ứng của buff chủ yếu dùng render của javafx làm nền
- Package **cache**: phụ trách việc cache dữ liệu trong RAM, do game chạy bằng DB trực tuyến nên gây trễ khi chơi, việc cache sẽ phần nào giúp game chạy nhanh hơn và tăng trải nghiệm người chơi.
- Package **cinematic**: một package khá đặc biệt, nó góp phần tạo nên một phần intro story rất điện ảnh.
- Package **Database**: Linh hồn của dữ liệu game, lưu trữ hệ thống cơ sở dữ liệu của game sử dụng dịch vụ server của AVIEN dựa trên MySQL, tạo các truy vấn lên DB để lưu lại tên người chơi room họ đang chơi trước khi thoát, số điểm, số vàng họ kiếm được, các vật phẩm họ đã trang bị trong shop, các slot vũ khí họ sử dụng, tự động đồng bộ dữ liệu mỗi 15s
- Package **engine**: Đây là phần quan trọng nhất của game nơi chứa class **GameWorld.java** nơi gọi toàn bộ logic render, update player, enemy, weapon, bullet, vật cản, pet, quản lí va chạm giữa các thành phần gọi các hiệu ứng của toàn bộ vòng lặp game. Đồng thời chứa class **Camera.java** quản lí góc nhìn của game với trung tâm là player, class **InputHandler.java** phụ trách việc tiếp nhận dữ liệu đầu vào để điều khiển game - đặc biệt là có thể chơi cảm ứng với touchpad.

- Package **entity**: quản lí các đối tượng chính **Player** và **Enemy** kế thừa từ **Entity** của game. Quản lí các loại hero người chơi có thể lựa chọn, mua của Shop . Đồng thời cũng quản lí các loại quái, logic quái tự điều khiển để đánh Player.
- Package **item**: quản lí các vật phẩm nhặt được trong màn chơi, trong đó có gold, gem, và buff
- Package **map**: phụ trách việc load map từ file json, gán các chỉ số trong tiltmap với các assets, quản lí các **Room** và logic đóng mở cửa khi hoàn thành mission đánh quái ở từng phòng
- Package **pet**: Quản lí pet, các logic về pet- người bạn đồng hành trợ giúp Player đánh quái và các con pet để mua trong shop.
- Package **ui**: Quản lí toàn bộ giao diện game từ *intro, login, register* đến *main menu shop, leaderboard, setting, HUD*. Đồng thời cũng đồng bộ dữ liệu với Database đặc trưng riêng cho từng người dùng. Trái tim là class **UIManager.java** gọi toàn bộ logic UI từ các file **controler** và các file giao diện **FXML** kết hợp với style trong các file **CSS** cùng với DB quản lí các giao diện chuyên biệt với từng tài khoản khác nhau như chơi lần đầu, đợi load dữ liệu, ...
- Package **utils**: Quản lí các tiện ích cơ bản như các hằng số, load âm thanh, DatabaseExcuter và vector2D
- Package **wapon**: quản lý hệ thống vũ khí, các loại vũ khí được load lên shop và logic chiến đấu của từng loại.
- Class **Main.java** là điểm khởi đầu (entry point) của ứng dụng Spirit Knight. Lớp này kế thừa **Application** của JavaFX và chịu trách nhiệm khởi tạo các thành phần chính cần thiết trước khi trò chơi bắt đầu hoạt động. Các trách nhiệm chính của lớp **Main** bao gồm:
 - Khởi tạo cửa sổ JavaFX và vùng render của game.
 - Khởi tạo hệ thống xử lý input.
 - Khởi tạo **GameWorld**.
 - Khởi tạo và liên kết **UIManager** với game.
 - Khởi tạo và điều khiển **GameLoop**.
 - Khởi tạo kết nối cơ sở dữ liệu trên một luồng riêng.
 - Quản lý quá trình lưu dữ liệu và giải phóng tài nguyên khi người chơi thoát game.

2.3 Class Main.java

2.3.1 Khởi tạo giao diện JavaFX

Khi ứng dụng được khởi chạy, JavaFX gọi phương thức **start(Stage stage)**. Đây là nơi lớp **Main** khởi tạo cửa sổ chính của trò chơi. Một đối tượng **Canvas** được tạo với kích thước mặc định được định nghĩa trong lớp **Constants**. Canvas đóng vai trò là bề mặt chính để render các thành phần của thế giới game.

```
Canvas canvas = new Canvas(  
    Constants.WINDOW_WIDTH,  
    Constants.WINDOW_HEIGHT  
);
```

```
GraphicsContext gc = canvas.getGraphicsContext2D();
```

Đối tượng `GraphicsContext` được lấy từ `Canvas` và được sử dụng bởi hệ thống render để vẽ các thành phần của trò chơi.

2.3.2 Khởi tạo hệ thống Input

Lớp `InputHandler` chịu trách nhiệm tiếp nhận các sự kiện đầu vào từ người chơi như bàn phím và các thao tác điều khiển được hỗ trợ bởi game. `InputHandler` được liên kết với `Scene` để nhận các sự kiện đầu vào và sau đó được truyền vào `GameWorld`.

2.3.3 Khởi tạo GameWorld và UIManager

Sau khi hệ thống input được thiết lập, `Main` khởi tạo đối tượng `GameWorld`.

```
GameWorld world = new GameWorld(inputHandler);
```

`GameWorld` đóng vai trò quản lý trạng thái và các thành phần chính của thế giới game. Tiếp theo, hệ thống giao diện được khởi tạo thông qua `UIManager`.

Việc liên kết `UIManager` với `GameWorld` cho phép giao diện truy cập các trạng thái cần thiết của game để cập nhật HUD và các thành phần giao diện khác.

2.3.4 Khởi tạo Game Loop

Lớp `Main` khởi tạo `GameLoop` để thực hiện vòng lặp chính của trò chơi. Mỗi vòng lặp nhận giá trị `delta`, biểu diễn khoảng thời gian giữa hai frame liên tiếp.

Trong mỗi frame, hệ thống thực hiện ba nhiệm vụ chính:

1. Xử lý input toàn cục.
2. Cập nhật trạng thái thế giới game.
3. Render thế giới và cập nhật HUD.

2.3.5 Khởi tạo cơ sở dữ liệu bất đồng bộ

Việc kiểm tra và khởi tạo cơ sở dữ liệu được thực hiện trên một luồng riêng thay vì trực tiếp trên `JavaFX Application Thread`. Thiết kế này giúp quá trình kết nối cơ sở dữ liệu không chặn `JavaFX Application Thread`, từ đó hạn chế tình trạng giao diện bị đứng trong quá trình khởi động game. Luồng này được thiết lập là `daemon thread` để không ngăn JVM kết thúc khi ứng dụng được đóng.

Chương 3

Hệ thống GameWorld

3.1 Tổng quan

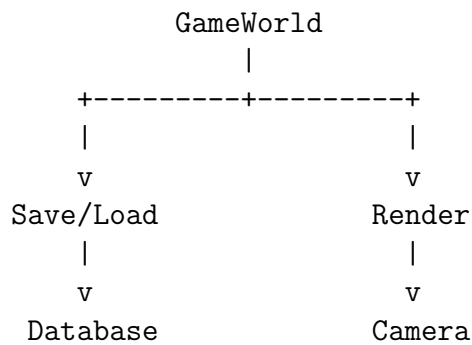
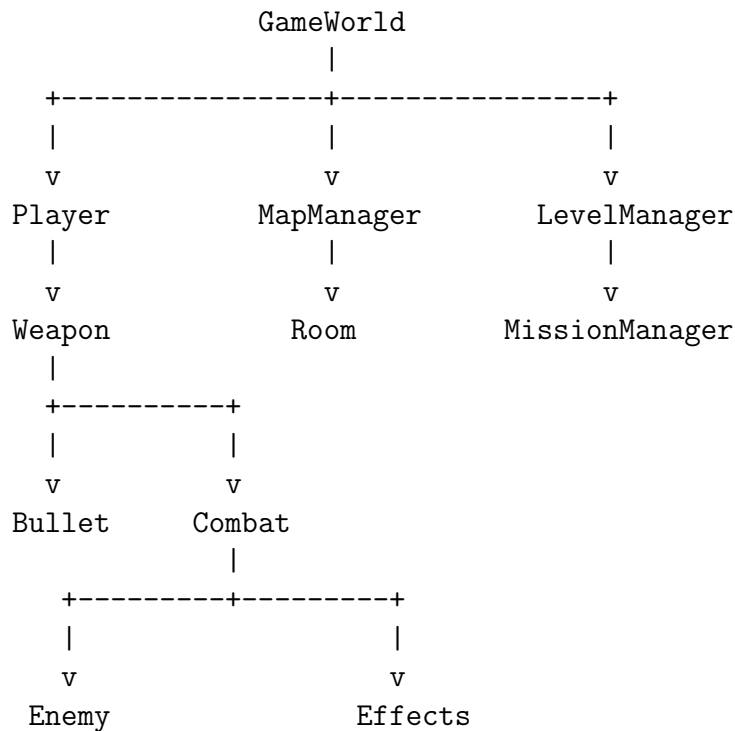
Lớp `GameWorld` thuộc package `com.soulknight.engine` và đóng vai trò là bộ điều phối trung tâm của thế giới trò chơi `Spirit Knight`. Khác với lớp `Main` chịu trách nhiệm khởi tạo ứng dụng, `GameWorld` quản lý trạng thái của một phiên chơi và điều phối hoạt động giữa các hệ thống gameplay. Các thành phần được quản lý bởi `GameWorld` bao gồm:

- Player và Pet.
- Enemy và quá trình spawn Enemy.
- Map, Room và Obstacle.
- Bullet và hệ thống va chạm.
- Weapon và loadout của lượt chơi.
- Item và quá trình thu thập vật phẩm.
- Buff và các hiệu ứng chiến đấu.
- Particle và các hiệu ứng hình ảnh.
- Mission và Reward.
- Camera và quá trình render thế giới.
- Game State.
- Save/Load dữ liệu người chơi.

Có thể xem `GameWorld` như lớp trung gian kết nối các subsystem gameplay với vòng lặp chính của trò chơi.

3.2 Kiến trúc tổng thể

Về mặt kiến trúc, `GameWorld` không trực tiếp cài đặt toàn bộ hành vi của từng đối tượng. Thay vào đó, lớp giữ tham chiếu đến các manager và entity, sau đó điều phối chúng trong quá trình update và render. Mô hình tổng quát có thể biểu diễn như sau:



Thiết kế này cho phép các hệ thống chuyên biệt như **MapManager**, **LevelManager**, **MissionManager** hay **CombatEffectManager** tập trung vào trách nhiệm riêng, trong khi **GameWorld** quyết định thời điểm chúng được cập nhật.

3.3 Các thành phần chính

3.3.1 Các Manager

GameWorld sở hữu nhiều manager phục vụ cho các hệ thống khác nhau của game.

3.4 Quản lý trạng thái trò chơi

Một trong những trách nhiệm quan trọng nhất của **GameWorld** là quản lý trạng thái hiện tại của trò chơi thông qua **GameState**. Các trạng thái được xử lý trong **GameWorld** bao gồm:

- INTRO

Thành phần	Trách nhiệm
LevelManager	Quản lý level và tiến trình của một lượt chơi.
MapManager	Quản lý bản đồ, tile, room, portal và dữ liệu không gian.
MissionManager	Quản lý nhiệm vụ của level hiện tại.
EnemyFactory	Khởi tạo các Enemy phù hợp với level.
ParticleManager	Quản lý các particle effect trong game.
EnemyBurnManager	Quản lý trạng thái Burn của Enemy.
CombatEffectManager	Quản lý các hiệu ứng chiến đấu đặc biệt.
ItemMagnetSystem	Điều khiển cơ chế hút Item về phía Player.
FloatingTextManager	Hiển thị damage, gold và các thông báo nổi trong game.

Bảng 3.1: Một số subsystem được GameWorld quản lý

- MAIN_MENU
- PLAYING
- PAUSED
- LEVEL_CLEAR
- REWARD_PICK
- GAME_OVER
- GAME_VICTORY

3.4.1 Chuyển đổi trạng thái

Việc chuyển trạng thái được tập trung trong phương thức `changeState()`. Khi trạng thái thay đổi, trạng thái input cũ được xóa nhằm tránh việc một thao tác bàn phím hoặc chuột từ state trước tiếp tục ảnh hưởng tới state mới. Sau đó `GameStateListener` được thông báo để các hệ thống khác, đặc biệt là UI, có thể phản ứng với trạng thái mới.

3.5 Chu trình cập nhật GameWorld

GameWorld trong mỗi frame. Nếu trò chơi đang ở trạng thái `PAUSED`, phương thức kết thúc ngay lập tức và toàn bộ gameplay ngừng cập nhật. Đối với các trạng thái còn lại, GameWorld sử dụng `switch` để chuyển việc xử lý đến logic tương ứng.

```

GameLoop
|
v
GameWorld.update()
|
+-- PAUSED -----> Stop Update
|

```

```

+-- INTRO -----> Input
|
+-- MAIN_MENU -----> Input
|
+-- PLAYING -----> updatePlaying()
|
+-- LEVEL_CLEAR ----> updateLevelClear()
|
+-- REWARD_PICK ----> updateRewardPick()
|
+-- GAME_OVER -----> Restart
|
+-- GAME_VICTORY ----> Restart

```

3.6 Cập nhật gameplay

Khi state hiện tại là **PLAYING**, GameWorld gọi phương thức `updatePlaying()`. Đây là một trong những phương thức điều phối quan trọng nhất của hệ thống gameplay. Quá trình update được thực hiện theo thứ tự tổng quát:

1. Cập nhật hiệu ứng spawn và death.
2. Cập nhật Player.
3. Cập nhật Pet.
4. Xác định Room hiện tại.
5. Cập nhật trạng thái Room.
6. Cập nhật Enemy.
7. Xử lý va chạm Player–Enemy.
8. Cập nhật Bullet.
9. Xử lý Obstacle bị phá hủy.
10. Cập nhật các Combat Effect.
11. Cập nhật trạng thái Burn.
12. Cập nhật Item Magnet.
13. Kiểm tra Player thu thập Item.
14. Kiểm tra Reward.
15. Thực hiện Auto Save.
16. Cập nhật Camera.

17. Kiểm tra trạng thái sống/chết của Player.

18. Cập nhật Particle và Floating Text.

Thứ tự update có ý nghĩa quan trọng vì nhiều subsystem phụ thuộc vào kết quả của subsystem trước đó.

3.7 Quản lý Player và Pet

GameWorld giữ tham chiếu đến Player hiện tại thông qua lời gọi đến class Player. Trong trạng thái PLAYING, Player được cập nhật trước nhiều hệ thống chiến đấu khác. Việc truyền `this` vào `player.update()` cho phép Player tương tác với dữ liệu của thế giới như Enemy, Bullet, Map và các subsystem gameplay.

3.7.1 Pet System

Loại Pet được lấy từ `PetSelectionManager` và instance thực tế được tạo thông qua `PetFactory`. Nhờ đó, hệ thống lựa chọn Pet và đối tượng Pet tồn tại trong game được tách biệt với nhau.

3.8 Hệ thống chiến đấu

GameWorld đóng vai trò điều phối nhiều thành phần của Combat System, bao gồm:

- Bullet.
- Slash Effect.
- Explosion Effect.
- Ion Explosion.
- Sound Wave.
- Burn Effect.
- Damage.
- Combat Buff.

3.8.1 Quản lý Bullet

Các projectile đang tồn tại được lưu trong:

```
private final List<Bullet> bullets = new ArrayList<>();
```

Trong mỗi frame, `updateBullets()` chịu trách nhiệm cập nhật vị trí và kiểm tra va chạm. Một Bullet có thể tương tác với:

- Wall.
- Obstacle.

- Enemy.
- Player.

Tùy loại vũ khí, Bullet còn có thể có các hành vi đặc biệt như phản xạ, xuyên mục tiêu, tạo explosion hoặc tạo sound wave.

3.8.2 Damage

Khi Bullet của Player va chạm Enemy, GameWorld tính lượng damage thực tế dựa trên HP trước và sau khi nhận sát thương. Mô hình xử lý có dạng:

```

Bullet
|
v
Collision
|
v
Enemy.takeDamage()
|
+----> Floating Damage Text
|
+----> Particle Effect
|
+----> Buff Notification

```

Việc sử dụng lượng damage thực tế giúp hệ thống Buff nhận đúng lượng damage đã gây ra thay vì chỉ sử dụng damage lý thuyết của projectile.

3.9 Hệ thống kỹ năng và hiệu ứng chiến đấu

GameWorld còn điều phối các kỹ năng đặc biệt của Player-được kích hoạt khi sử dụng các Buffs Ví dụ, hệ thống hiện tại bao gồm:

- Dragon Breath.
- Holy Nova.
- Chain Lightning.
- Death Explosion.
- Ion Explosion.

Đối với Dragon Breath, GameWorld xác định Enemy nằm trong vùng hình nón trước mặt Player. Damage được xử lý trực tiếp bởi logic gameplay, trong khi animation chỉ đóng vai trò biểu diễn trực quan. Cách tách gameplay logic và visual effect này giúp kết quả combat không phụ thuộc vào thời gian chạy animation.

3.10 Quản lý Item

GameWorld sử dụng `ItemMagnetSystem` để cập nhật chuyển động hút Item về phía Player. Sau đó `updateItemCollection()` kiểm tra va chạm giữa Player và Item. Các loại Item hiện được xử lý bao gồm:

- Gold.
- Gem.
- Buff.
- Energy Crystal.

Sau khi Item được thu thập, GameWorld cập nhật dữ liệu tương ứng, tạo visual feedback và loại Item khỏi danh sách active.

3.11 Hệ thống Room và Reward

GameWorld theo dõi Room mà Player đang đứng. Trạng thái Room ở frame trước cũng được lưu lại để GameWorld có thể phát hiện sự thay đổi trạng thái của phòng. Sau khi một phòng chiến đấu được hoàn thành, hệ thống có thể mở `RewardPicker`. Reward hiện tại cho phép Player lựa chọn giữa:

- Gold.
- Weapon mới.

Mỗi phòng chỉ được nhận reward một lần trong cùng một lượt chơi.

3.12 Hệ thống vũ khí trong một lượt chơi

GameWorld quản lý hai slot vũ khí riêng của lượt chơi. Khi bắt đầu một run, loadout được snapshot từ `WeaponSelectionManager`. Nếu slot không có dữ liệu hợp lệ, hệ thống sử dụng vũ khí mặc định:

- Slot 1: BLASTER.
- Slot 2: OLD_SWORD.

Thiết kế này tạo ra sự phân biệt giữa:

```
Shop Loadout
|
| Start Run
v
Run Loadout
|
+---- Slot 1
|
+---- Slot 2
```

Do đó, việc nhận Weapon Reward trong dungeon có thể thay đổi vũ khí của run hiện tại mà không nhất thiết thay đổi lựa chọn vũ khí được lưu trong Shop.

3.13 Khởi tạo một lượt chơi

Khi bắt đầu một lượt chơi mới, GameWorld thực hiện quá trình khởi tạo thông qua `startNewRun()` và `loadCurrentLevel()`. Quá trình load level bao gồm:

1. Khởi tạo MapManager.
2. Load Obstacle từ Tile Map.
3. Xác định spawn point.
4. Khởi tạo hoặc di chuyển Player.
5. Khôi phục dữ liệu Save nếu có.
6. Khởi tạo Pet.
7. Tạo Spawn Effect.
8. Trang bị Weapon.
9. Xóa Entity và Effect của level trước.
10. Tạo Mission.
11. Khởi tạo Item đặc thù của level.
12. Spawn Boss nếu level hiện tại là Boss Level.

3.14 Render thế giới

Phần render gameplay chính được thực hiện bởi `renderWorld()`.

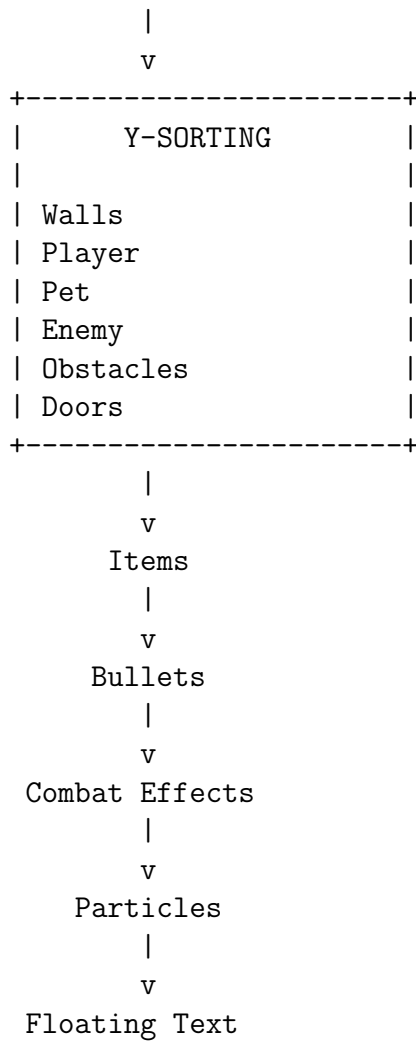
3.14.1 Y-Sorting

Spirit Knight sử dụng góc nhìn giả 2.5D. Vì vậy, thứ tự render không thể chỉ phụ thuộc vào loại Entity. GameWorld tạo một danh sách các đối tượng render và gán cho mỗi đối tượng một giá trị độ sâu dựa trên tọa độ Y. Những đối tượng có vị trí thấp hơn trên màn hình sẽ được render sau, tạo cảm giác chúng nằm phía trước đối tượng có tọa độ Y nhỏ hơn.

3.14.2 Render Pipeline

Pipeline render tổng quát có thể biểu diễn:

```
Dynamic Background
    |
    v
  Floor
    |
    v
Obstacle Shadows
```



3.15 Camera

GameWorld sở hữu một đối tượng **Camera** dùng để chuyển đổi tọa độ thế giới sang tọa độ màn hình. Trong quá trình gameplay, camera liên tục follow Player: Ngoài ra, GameWorld kiểm tra object có nằm trong vùng camera hay không trước khi render. Điều này giúp tránh thực hiện các draw call không cần thiết đối với object nằm ngoài viewport.

3.16 Hệ thống Save và Load

GameWorld tích hợp trực tiếp với hệ thống persistence thông qua:

- PlayerSave.
- PlayerSaveDAO.
- PlayerSaveMapper.
- ShopDAO.
- UserSession.

3.16.1 Database Worker

Các thao tác database được chuyển sang một luồng `ExecutorService` riêng.

```
private final ExecutorService databaseExecutor =  
    Executors.newSingleThreadExecutor(runnable -> {  
        Thread thread = new Thread(  
            runnable ,  
            "soul-knight-database-worker "  
        );  
  
        thread.setDaemon(true);  
        return thread;  
    });
```

Mục tiêu của thiết kế này là tránh thực hiện các thao tác I/O database trực tiếp trên game thread, từ đó giảm nguy cơ gây khựng hoặc đứng game.

3.16.2 Cloud Save

Phương thức `saveGameAsync()` chuyển trạng thái hiện tại của `GameWorld` thành `PlayerSave` thông qua `PlayerSaveMapper`. Sau đó dữ liệu được lưu bằng `PlayerSaveDAO` trên database worker.

3.16.3 Cloud Load

Quá trình load được thực hiện bất đồng bộ thông qua `loadGameAsync()`. Dữ liệu lấy từ database được lưu tạm trong:

```
private PlayerSave pendingPlayerSave;
```

Sau khi Player và level đã được khởi tạo, dữ liệu này mới được áp dụng trở lại `GameWorld`. Cách thực hiện này tránh việc restore dữ liệu vào Player trước khi Player thực sự tồn tại.

3.17 Auto Save

`GameWorld` định nghĩa khoảng thời gian Auto Save:

```
private static final double AUTO_SAVE_INTERVAL = 30.0;
```

Trong quá trình gameplay, bộ đếm thời gian được cập nhật theo `deltaSeconds`. Khi đạt đến khoảng thời gian quy định, `GameWorld` yêu cầu lưu trạng thái hiện tại. Việc save thực tế vẫn được chuyển sang database worker để không làm gián đoạn game loop.

3.18 Quản lý Gold và Gem

`GameWorld` phân biệt tài nguyên thu được trong run với tài nguyên đã được đồng bộ vào tài khoản. Các biến: lưu lượng tài nguyên đang chờ đồng bộ. Khi Player nhận Gold hoặc Gem, giá trị tương ứng được cộng vào pending bank. Sau đó `syncBankRewardsAsync()` chuyển lượng tài nguyên này sang database. Nếu quá trình database thất bại, lượng Gold và Gem chưa đồng bộ được trả lại vào pending bank để có thể thử lại ở lần sau. Cơ chế này giúp giảm nguy cơ mất reward khi xảy ra lỗi kết nối database.

3.19 Quản lý đồng thời

Do GameWorld vừa hoạt động trên game thread vừa giao tiếp với database worker, một số trạng thái cần được bảo vệ khỏi việc truy cập đồng thời. Các đối tượng `AtomicBoolean` được sử dụng để ngăn nhiều thao tác Save, Load hoặc Bank Sync chạy đồng thời. Ngoài ra, dữ liệu reward đang chờ đồng bộ được bảo vệ bởi một lock riêng. Thiết kế này đặc biệt quan trọng vì database worker hoạt động trên thread khác với gameplay.

3.20 Quá trình Shutdown

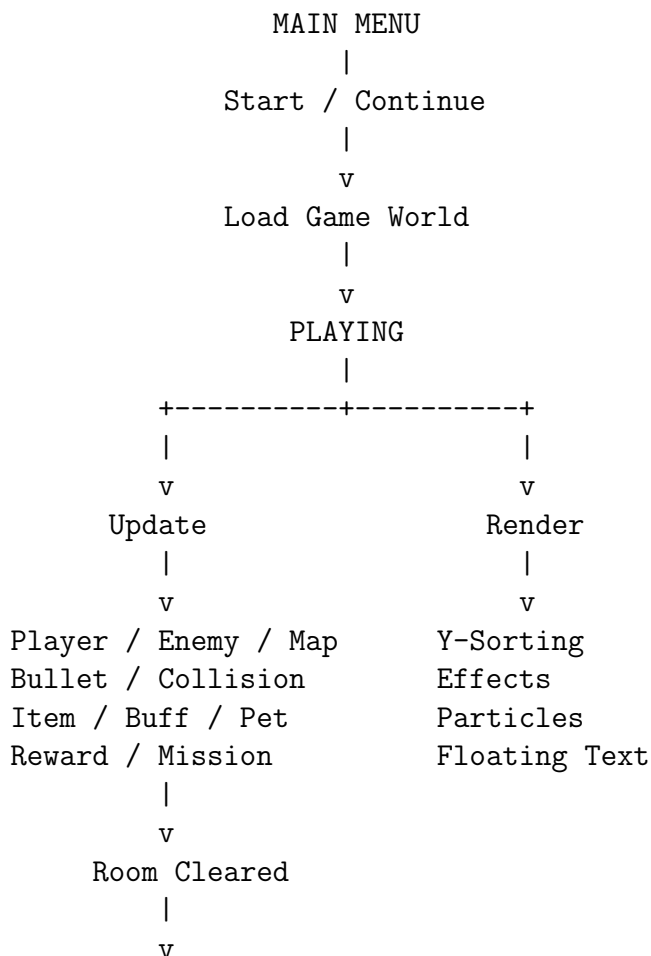
Khi ứng dụng chuẩn bị đóng, phương thức `shutdown()` được sử dụng để kết thúc các tài nguyên thuộc GameWorld. Quá trình này bao gồm:

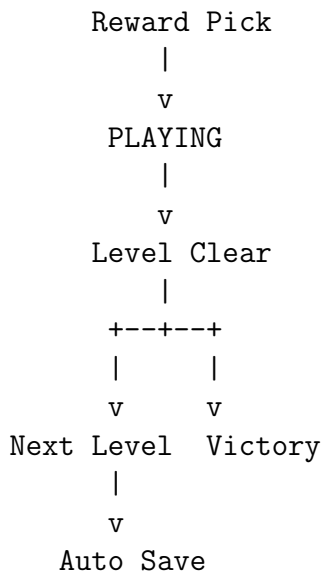
1. Đồng bộ Gold và Gem còn tồn đọng.
2. Yêu cầu database executor dừng hoạt động.

Điều này phối hợp với shutdown handler của lớp `Main` để hạn chế mất dữ liệu khi người chơi đóng game.

3.21 Luồng hoạt động tổng quát

Có thể tóm tắt vòng đời gameplay được điều phối bởi `GameWorld` như sau:





3.22 Kết luận

GameWorld là một trong những lớp trung tâm của kiến trúc Spirit Knight. Lớp kết nối Game Loop với hầu hết các subsystem gameplay và chịu trách nhiệm duy trì trạng thái của một lượt chơi. Thông qua GameWorld, quá trình cập nhật Player, Enemy, Bullet, Room, Item, Pet, Mission, Reward, Camera, Render và Save/Load được thực hiện theo một trình tự thống nhất trong mỗi frame. Vì vậy, có thể xem **GameWorld** là lớp điều phối gameplay chính của Spirit Knight.